

Bloom Paxos: Efficient State-Based Consensus for Geo-Distributed Systems

Raj Kumar Pandey

Computer Science and Engineering
The Pennsylvania State University
University Park, PA, USA
rzp5412@psu.edu

Viveck R. Cadambe

School of Electrical and Computer Engineering
Georgia Institute of Technology
Atlanta, GA, USA
viveck@gatech.edu

Bhuvan Urgaonkar

Computer Science and Engineering
The Pennsylvania State University
University Park, PA, USA
buu1@psu.edu

Abstract—Strong consistency is desirable for programmers and for many applications in distributed systems, but it tends to come at a high cost. In geo-distributed systems, data transfer is a significant contributor to that cost. This paper presents Bloom Paxos, a novel consensus algorithm that offers a flexible trade-off between data accuracy and system cost. Bloom Paxos replaces the replicated command log of standard Paxos-family protocols with direct replication of the state machine’s state, using a Bloom filter to suppress duplicate command execution across leader changes. We describe the algorithm, prove that under an ideal Bloom filter, it applies each client command at most once, in a single order consistent with real time, and bound the false-positive rate of a practical Bloom filter. An experimental evaluation shows that Bloom Paxos reduces data transfer cost by 43% compared to Raft in a geo-distributed deployment with periodic quorum changes, while maintaining comparable latency.

I. INTRODUCTION

Distributed systems underpin modern computing by distributing tasks across interconnected nodes to achieve scalability, fault tolerance, and performance. Building such systems over consensus has driven the development of protocols such as Paxos [1], [2], Raft [3], and their many variants, for example [4]–[6]. To reduce the latency and storage cost of these protocols, researchers have explored intelligent data and leader placement [7], protocol-level modifications [4], [8], and erasure coding [9]–[11]. In this paper, we explore a new dimension along the cost axis: applications that can tolerate a small, controlled error in the replicated value. Prior work has examined related probabilistic relaxations of consistency, including bounded staleness in eventually consistent systems [12] and probabilistic quorum systems that relax quorum intersection, allowing a small probability of inconsistency in exchange for improved fault tolerance and availability [13].

Fault-tolerant replicated state machines [14] are conventionally built on a replicated command log, using one of two approaches. In the first, exemplified by Multi-Paxos [1], each log entry is decided by a separate instance of basic, *single-instance* Paxos, which agrees on a single value. Because these instances run independently and may run concurrently, a later entry can be decided before an earlier one. The log can then have *holes*: for example, entry 6 is decided, but entry 5 is not.

Since a node applies commands to its state machine in log order, it cannot apply entry 6 until it learns entry 5.

In the second, exemplified by Raft [3] and Sequence Paxos [15], a leader sends entries to its followers and a follower acknowledges an entry only after receiving all preceding ones, so logs are hole-free by construction. The two approaches differ in how they order entries, but both replicate the command log. Consequently, both pay the same cost on a quorum change: a joining or recovering node must obtain every log entry it lacks, which entails substantial data transfer that grows with log length.

A third approach eliminates the log entirely. Rather than agreeing on a log of commands, as in all of the protocols above, Paxos-based consensus agrees directly on the *state* of the state machine. The benefit is that a joining or recovering node needs only the current state, rather than the log that produced it. Thus, the amount of data transferred depends on the size of the state, not on how many commands have been executed. This third approach has been explored in CASPaxos [16] and RMWPaxos [17]; however, these works suffer from a significant *command duplication* issue, which is explained next and can render the state machine incorrect. Our contribution is a novel state-based protocol, *Bloom Paxos*, that addresses the command duplication drawback via Bloom filters, thus bridging an important gap between state-based protocols and practical implementation.

Recall that in single-instance Paxos, a proposer proposes a value, and the value is *committed* (i.e., consensus is reached) once a majority of acceptors *accept* the proposal. When Paxos is used to replicate a log (e.g., via Multi-Paxos), the proposed value is a command associated with a particular log entry; in the state-based approach, the proposer instead applies the command on receipt and proposes the resulting state. The shortcoming of this approach is that the proposer ships only the current state, not the command history that produced it. Different proposers can therefore propose the same command, causing it to be applied more than once. For example, if a proposer crashes after committing a command but before responding to the client, the client retries with a new proposer, and the command is executed again. Including the full command log in each message would resolve this, but it would erase the very cost savings described above.

Bloom Paxos differs from CASPaxos and RMWPaxos in how it detects duplicate retries. CASPaxos keeps no record of past requests, so it performs no duplicate detection. RMWPaxos prevents a proposer from re-applying its own request if an earlier attempt succeeded without its knowledge; because its request identifiers are generated by the proposer, a client retry through a different proposer can still execute the command twice.

Bloom Paxos detects duplicates as follows. The client attaches an identifier to its command and reuses it on every retry. Whenever a proposer applies a command, it inserts that identifier in a Bloom filter, and this filter is always transmitted with the state. So any new leader receives the Bloom filter along with the state, and consults it before applying a command: if the identifier is already there, the command has run before and is skipped. The Bloom filter’s accuracy is tunable, yielding a controlled trade-off between accuracy and cost (Sections II and III).

We prove that Bloom Paxos executes every client command at most once under an ideal Bloom filter (Appendix A), and we implement it and evaluate it against Raft, which we take as a representative log-based protocol. In a geo-distributed deployment with periodic quorum changes, Bloom Paxos reduces data-transfer cost by 43% at comparable request latency, and a distributed counter running on it shows no errors over 10 million operations (Section IV).

Bloom Paxos is a good choice for applications where state machine size does not grow with command application, such as distributed counters, gauges, configuration registers, leases, lock ownership, and similar coordination metadata. Bloom Paxos is also advantageous when quorums or leaders change frequently, because a joining node needs only the current state instead of an ever-growing log. Conversely, applications whose state grows without bound (for example, a key-value store accumulating entries), or that need a command log for purposes beyond consensus such as auditing, debugging, or event replay, remain better served by log-based protocols.

II. BACKGROUND AND RELATED WORK

Consensus algorithms: Distributed consensus algorithms such as Raft [3] and Multi-Paxos replicate logs of commands across the system; the consensus module ensures these logs are identical at every node. Each node then feeds its log into a local state machine, and so long as all nodes start from the same initial state and apply the same commands, they all converge to the same final state. Log-based consensus underpins widely deployed systems such as Chubby [18], Spanner [19], and Megastore [20], in all of which the log must be pruned, snapshotted, and compacted to bound memory; Chandra et al. [21] report this log management as one of the primary difficulties of deploying Paxos in practice. Algorithms such as CASPaxos [16] and RMWPaxos [17] instead operate directly on the state, leaving no log to maintain.

Bloom filter: A Bloom filter [22] is a space-efficient probabilistic data structure used to test whether an element is a member of a set. While Bloom filters may return false

positives, they never produce false negatives. The following results are standard [22]. A filter with m bits and k hash functions holding n elements reports a non-member as present with probability

$$p_{1bf} = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k,$$

and with the optimal number of hash functions $k = \frac{m}{n} \ln 2$, the filter size, in number of bits, required to achieve a target false positive probability p_{1bf} at n elements is

$$m = -\frac{n \ln(p_{1bf})}{(\ln 2)^2}.$$

As more elements are inserted beyond the design capacity n , the false positive probability rises, and the filter eventually becomes ineffective. A standard Bloom filter supports no deletion: clearing the bits of one element would also clear bits it shares with others, introducing false negatives. Counting Bloom filters [23], [24] restore deletion by replacing each bit with a counter, quadrupling the space for the standard four-bit counters. Cuckoo filters [25] also support deletion, at comparable space and with faster lookups, while Scalable Bloom filters [26] add capacity by growing on demand. These approaches do not meet our requirements. To support deletion, we need to store metadata about each element to identify it, effectively maintaining the insertion history that state-based replication is designed to avoid. Scalable Bloom filters are not suitable because they grow in size, which again requires more space. Hence, we need an option that does not require information about past added elements and stays fixed in size.

Two-Bloom-filter design: We employ a two-Bloom-filter (2-BF) design, similar in spirit to prior work [27], which periodically discards and rebuilds filters, with the following characteristics: (i) an element that is among the most recent n insertions of the stream is always reported present, the 2-BF has no false negatives within this window; (ii) each element is retained for at least n and at most $2n - 1$ subsequent insertions and is then deliberately forgotten, so a query for an older element may return a false negative; and (iii) an element that was never inserted is reported present with probability at most p_{2bf} , a configurable bound derived below.

The 2-BF system consists of two Bloom filters with identical parameters, denoted as BF_{new} and BF_{old} . Insertions are grouped into *epochs* of n insertions each. During an epoch, incoming elements are inserted into BF_{new} . Once n elements have been inserted, the filters are rotated: BF_{new} is promoted to become BF_{old} , the previous BF_{old} is discarded, and a fresh empty BF_{new} is initialized. An element inserted as the i -th insertion of an epoch thus remains present for the remaining $n - i$ insertions of its epoch and for all n insertions of the following epoch, which establishes characteristics (i) and (ii) above. Membership queries in the 2-BF system are processed by checking both BF_{new} and BF_{old} . If either filter reports an element as present, the query returns a positive result; otherwise, the element is declared absent.

Let $FPP(BF_{new})$ and $FPP(BF_{old})$ denote the false positive probabilities of the new and old Bloom filters, respectively. A false positive in the 2-BF system occurs if at least one filter returns a positive result for a non-member element. In 2-BF, each element is inserted into only one of them (BF_{new}), so the two filters hold disjoint elements and, under the standard uniform-hashing assumption, their false positives are independent. The false positive probability of the 2-BF system is therefore

$$FPP(2BF) = 1 - (1 - FPP(BF_{new}))(1 - FPP(BF_{old})).$$

Each filter's false positive probability increases with the number of elements it holds, which varies over an epoch: BF_{new} holds between 0 and n elements, while BF_{old} holds exactly n (it is rotated only when full; it is empty during the very first epoch). Our analysis is therefore worst-case, taken at the point where both filters hold n elements. Let p_{1bf} denote the false positive probability of a single filter holding its maximum of n elements. Since $FPP(BF_{new}) \leq p_{1bf}$ and $FPP(BF_{old}) \leq p_{1bf}$ at all times, the system-level false positive probability satisfies

$$FPP(2BF) \leq 1 - (1 - p_{1bf})^2 = p_{2bf},$$

where p_{2bf} denotes this worst-case bound.

Given the target bound p_{2bf} , we derive the required worst-case per-filter false positive probability $p_{1bf} = 1 - \sqrt{1 - p_{2bf}}$ by solving the equation above, and then size each filter for n elements using the standard formulas stated at the beginning of this section. Since each filter holds at most n elements at any time, this guarantees the bound p_{2bf} throughout every epoch.

III. ALGORITHM DESIGN AND ANALYSIS

We present the Bloom Paxos algorithm and a first-order analysis of its performance and cost.

A. High-Level Description

Paxos is most often presented as a single-instance consensus algorithm (e.g., [1]), and replicated state machines are typically built on top using *Multi-Paxos*, which invokes single-instance Paxos as a black box. Following the spirit of Sequence Paxos [15] and Raft [3], our work presents a state machine replication algorithm built directly on the core Paxos technique. In [3], [15], the transmitted values are commands, and servers use consensus to agree on a stored command log. In Bloom Paxos, by contrast, the transmitted value is the *state* of the state machine itself, and servers use consensus to replicate and agree on the state directly. A second key difference is that Bloom filters replace command sequences or logs in transmitted messages. Figure 1 illustrates the resulting data-transfer savings over Raft, which we analyze in Section IV-C.

Client Protocol: The client generates a unique operation identifier, *opid*. On invocation of a command C , the client dispatches a $\langle C, opid \rangle$ message to the server it *believes* is the current leader (the server designated by the oracle to drive the protocol; see the Server Protocol below), and awaits the

leader's response with the result of executing C . This belief is only a hint and need not be current or correct. A non-leader server ignores the client command. If no response is received within a pre-defined timeout (e.g., due to stale leader information or a leader failure), the client may resend its command, possibly to a new leader. Every retry carries the *same opid* as the original invocation; this is what enables the duplicate detection performed in Phase 2 below.

Server Protocol: Algorithms 1 and 2 give the complete server protocol; the description below walks through them by line number. The protocol has two phases, *prepare* and *accept*. As with other Paxos-family algorithms, under a stable leader, each command is applied to the state machine using only the *accept* phase. We assume a leader-election oracle, which can be implemented via standard techniques (see, e.g., [15]). The oracle generates monotonically increasing round numbers (r) and server IDs (L) and sends the message $\langle Leader, L, r \rangle$ to the nodes. Bloom Paxos's safety properties rely on a single uniqueness property: for any two oracle messages $\langle Leader, L, r \rangle$ and $\langle Leader, L', r' \rangle$, we have $L = L'$.

Server state: As in Paxos, every server maintains `promised_id`, `accepted_id`, and `accepted_value`, together with the highest round it has seen and its state—leader (in the prepare or accept phase) or follower (Alg. 1, lines 2–5). IDs are (round, cmd_index) pairs, compared lexicographically.¹ Initially `promised_id` = `accepted_id` = $\perp$, where $\perp$ is a distinguished value smaller than every ID (Alg. 1, line 4). This ordering convention ensures that a server that has not yet promised or accepted anything passes the promise and acceptance checks of Phases 1 and 2 (Alg. 1, line 24; Alg. 2, line 16), and that `accepted_id` = $\perp$ in a Phase-1 ack marks a server that has accepted no proposal. The command index is a non-negative integer that starts at 0 and is incremented *before* each proposal (Alg. 2, line 12); a round's prepare therefore uses index 0, while its accepts use indices 1, 2, ..., so every accept ID in a round strictly exceeds that round's prepare ID = (round, 0). The one non-standard piece of state is the Bloom filter BF (Alg. 1, line 6): concretely, the 2-BF of Section II, whose two component filters are stored, queried, and transmitted together. Five further variables are used only by a leader and are reset when a server becomes one (Alg. 1, lines 8–12): the FIFO queue `cmd_queue` of client commands awaiting proposal, the counter `cmd_index` of proposals made in the current round, the set `Placks` of Phase 1 responses (i.e., responses to prepare messages), `pending`, which holds the proposal currently awaiting acknowledgment, or $\perp$ (NULL) when no proposal is outstanding, and `committed_value`, the latest state the leader has committed in its current round, or $\perp$ if none.

Phase 1 (prepare), Algorithm 1: Upon receiving an oracle message $\langle Leader, L, r \rangle$ in which r exceeds the local round, a server s either becomes leader or becomes a follower.

¹Comparison rule, $ID_1 > ID_2$: either $ID_1.round > ID_2.round$, or $ID_1.round = ID_2.round$ and $ID_1.cmd_index > ID_2.cmd_index$.

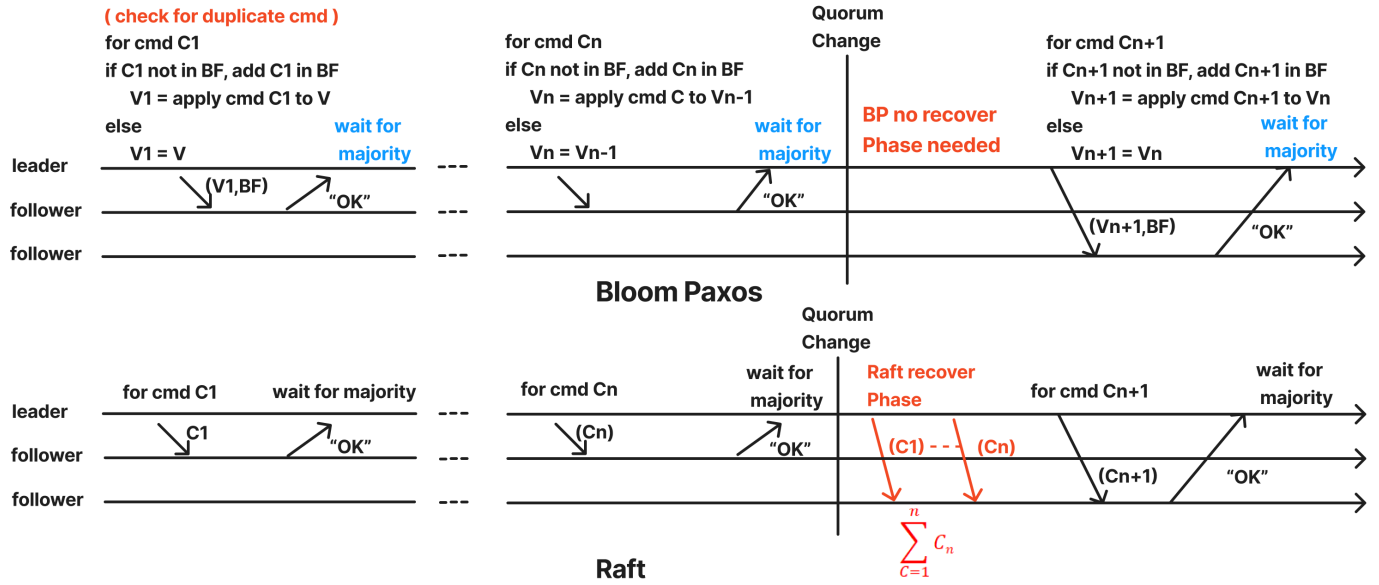


Fig. 1. Illustration of the data-transfer savings of Bloom Paxos over Raft.

If $s = L$, it resets its leader-only state (`cmd_queue`, `cmd_index`, `Placks`, `pending`, `committed_value`) and broadcasts a *prepare* carrying $ID = (r, 0)$ (index 0, reserved for prepares as noted above) to all servers, including itself (lines 13–19); if $s \neq L$, it records the new round and becomes a follower (line 21). Stale oracle messages are ignored (line 15), so the prepare phase runs at most once per round. Every server, the leader included, handles a *prepare* with the standard acceptor check (lines 24–26): if its `promised_id` is strictly smaller than the prepare’s ID , it sets its `promised_id` to the prepare’s ID and returns an ack carrying its $\langle \text{accepted_value}, \text{accepted_id}, \text{BF} \rangle$; otherwise it ignores the message (Alg. 1, line 28).

A leader collects acks only while its own `promised_id` is still the prepare’s $ID = (\text{round}, 0)$ (Alg. 1, line 30); once it has promised or accepted anything with a higher ID , it discards further acks for this round. Since every accept carries a command index of at least 1, this check also guarantees that the leader has accepted no proposal since answering its own prepare, so the state it reported in its own ack is still current. Once acks from a majority of servers (including the leader’s own) have arrived, the leader picks, among these acks, the one carrying the highest `accepted_id`, and replaces its local `accepted_value`, `accepted_id`, and, unlike single-instance Paxos, `BF` with the values in that ack; it then enters the accept phase (Alg. 1, lines 32–37).

Phase 2 (accept), Algorithm 2: Independently of the phase, a server acting as leader, in *either* the prepare or the accept phase, appends each client command $\langle C, \text{opid} \rangle$ it receives to `cmd_queue`, a first-in-first-out queue of commands awaiting proposal (Alg. 2, lines 1–3). Commands are appended unconditionally; duplicates are not removed here, but are detected later by the Bloom-filter check. The remaining steps run in the accept phase. The leader proposes

one command at a time: `pending` holds the proposal currently awaiting a majority, or $\perp$ when none is outstanding. Whenever `pending = \perp` and its own `promised_id` is still $(\text{round}, \text{cmd_index})$, that is, the ID of its most recent proposal of this round, the leader dequeues the next command (Alg. 2, lines 5–6) and executes the following steps.

To prevent duplicate execution, it first queries `BF` for the command’s *opid* (Alg. 2, line 7):

- If *opid* is not present in `BF`, the command is new: the leader applies command C to the current state `accepted_value` to obtain the proposed state n_{val} and inserts *opid* into the Bloom filter `BF`, rotating the 2-BF if the per-epoch capacity is reached (line 8). Since every accept message carries the entire updated Bloom filter and each follower replaces its own Bloom filter with the received one (Alg. 2, lines 15–19), rotations propagate consistently to all nodes.
- If *opid* is present in Bloom filter `BF`, the command has already been executed: the leader skips re-execution and proposes the *unchanged* state and Bloom filter (Alg. 2, line 10), preserving at-most-once execution (Section III-B). This case arises when a previous leader crashed after committing the command but before responding to the client, prompting the client to retry with the same *opid*.

The leader then increments `cmd_index`, forms $ID = (\text{round}, \text{cmd_index})$, sets `pending` to $\langle ID, n_{val}, \text{client} \rangle$, and sends $\langle \text{accept}, ID, n_{val}, \text{BF}_{new} \rangle$ to all servers, itself included (Alg. 2, lines 12–14). Each server—the leader included—applies an *acceptance check* to the accept message: it accepts if its `promised_id` $\leq ID$. On passing the check, the server replaces its local `accepted_value` with the message’s value n_{val} and its `BF` with the message’s filter, sets `promised_id` = `accepted_id` = ID , and replies $\langle \text{accepted}, ID \rangle$ (Alg. 2, lines 16–19). If instead the leader’s

Algorithm 1 Bloom Paxos at server s : state and Phase 1 (prepare)

```
1: Persistent state at every server:
2:   round  $\leftarrow 0$   $\triangleright$  integer: highest round seen
3:   state  $\leftarrow$  follower  $\triangleright$ 
4:    $\in \{(\text{leader}, \text{prepare}), (\text{leader}, \text{accept}), \text{follower}\}$ 
5:   promised_id  $\leftarrow \perp$ ; accepted_id  $\leftarrow \perp$   $\triangleright$  IDs: (round, cmd_index) pairs or  $\perp$ ;  $\perp <$  every ID
6:   accepted_value  $\leftarrow s_0$   $\triangleright$  application state;  $s_0$  is the initial state
7:   BF  $\leftarrow$  empty 2-BF  $\triangleright$  Bloom filter pair of Section II
8: Leader-only state (reset on becoming leader):
9:   cmd_queue  $\leftarrow \langle \rangle$   $\triangleright$  FIFO queue of  $\langle C, \text{opid} \rangle$  pairs
10:  cmd_index  $\leftarrow 0$   $\triangleright$  integer: commands proposed in the current round
11:  Placks  $\leftarrow \emptyset$   $\triangleright$  set of  $\langle \text{val}, \text{aid}, \text{bf} \rangle$  triples
12:  pending  $\leftarrow \perp$   $\triangleright$  in-flight proposal  $\langle id, n_{\text{val}}, \text{client} \rangle$ , or  $\perp$  if none
13:  committed_value  $\leftarrow \perp$   $\triangleright$  latest state committed in this round, or  $\perp$ 
14: upon  $\langle \text{Leader}, L, r \rangle$  from the oracle do
15:   if  $r \leq \text{round}$  then
16:    ignore  $\triangleright$  stale oracle message
17:   else if  $s = L$  then
18:    round  $\leftarrow r$ ; state  $\leftarrow (\text{leader}, \text{prepare})$ 
19:    cmd_queue  $\leftarrow \langle \rangle$ ; cmd_index  $\leftarrow 0$ ; Placks  $\leftarrow \emptyset$ ; pending  $\leftarrow \perp$ ; committed_value  $\leftarrow \perp$ 
20:    send  $\langle \text{prepare}, (r, \text{cmd\_index}) \rangle$  to all servers, including  $s$  itself  $\triangleright$  cmd_index = 0 here (just reset)
21:   else
22:    round  $\leftarrow r$ ; state  $\leftarrow$  follower
23:   end if
24: upon  $\langle \text{prepare}, id \rangle$  from server  $L$  do  $\triangleright$  every server, including  $L$  itself
25:   if promised_id  $< id$  then
26:    promised_id  $\leftarrow id$ 
27:    send  $\langle \text{ack}, id, \text{accepted\_value}, \text{accepted\_id}, \text{BF} \rangle$  to  $L$ 
28:   else
29:    ignore  $\triangleright$  if  $s = L$ , the leader abandons round  $id.\text{round}$ 
30:   end if
31: upon  $\langle \text{ack}, id, \text{val}, \text{aid}, \text{bf} \rangle$  with  $id = (\text{round}, 0)$  and promised_id = (round, 0), while state = (leader, prepare) do
32:   Placks  $\leftarrow \text{Placks} \cup \{(\text{val}, \text{aid}, \text{bf})\}$ 
33:   if  $|\text{Placks}| = F+1$  then
34:     $(\text{val}^*, \text{aid}^*, \text{bf}^*) \leftarrow$  element of Placks with the highest aid
35:    if  $\text{aid}^* \neq \perp$  then
36:     accepted_value  $\leftarrow \text{val}^*$ ; accepted_id  $\leftarrow \text{aid}^*$ ; BF  $\leftarrow \text{bf}^*$ 
37:   end if
38:   state  $\leftarrow (\text{leader}, \text{accept})$ 
39: end if
```

own check fails, a higher round has meanwhile reached it and advanced its promised_id past ID , so the proposal can no longer gather a majority and the leader discards it (Alg. 2, line 21); it stops driving the current round and adopts the newer round—as leader or follower—once it processes the corresponding oracle message. The first accept of a round carries $ID = (\text{round}, 1)$, which exceeds the round’s prepare $ID = (\text{round}, 0)$, so it passes this check at every server

Algorithm 2 Bloom Paxos at server s : Phase 2 (accept) and reads

```
1: upon  $\langle C, \text{opid} \rangle$  from a client do
2:   if state  $\in \{(\text{leader}, \text{prepare}), (\text{leader}, \text{accept})\}$  then
3:    append  $\langle C, \text{opid} \rangle$  to cmd_queue
4:   end if
5:   upon state = (leader, accept) and pending =  $\perp$  and cmd_queue  $\neq \langle \rangle$  and promised_id = (round, cmd_index) do  $\triangleright$  one proposal in flight at a time
6:     $\langle C, \text{opid} \rangle \leftarrow \text{dequeue}(\text{cmd\_queue})$ 
7:    if  $\text{opid} \notin \text{BF}$  then
8:      $n_{\text{val}} \leftarrow \text{apply}(C, \text{accepted\_value})$ ;  $\text{BF}_{\text{new}} \leftarrow \text{BF} \cup \{\text{opid}\}$   $\triangleright$  rotate the 2-BF if the epoch capacity is reached
9:    else
10:      $n_{\text{val}} \leftarrow \text{accepted\_value}$ ;  $\text{BF}_{\text{new}} \leftarrow \text{BF}$   $\triangleright$  duplicate detected: do not re-execute  $C$ 
11:   end if
12:   cmd_index  $\leftarrow \text{cmd\_index} + 1$ ;  $id \leftarrow (\text{round}, \text{cmd\_index})$ 
13:   pending  $\leftarrow \langle id, n_{\text{val}}, \text{the client of } C \rangle$ ; P2accepts  $\leftarrow \emptyset$   $\triangleright$  P2accepts: set of servers
14:   send  $\langle \text{accept}, id, n_{\text{val}}, \text{BF}_{\text{new}} \rangle$  to all servers, including  $s$  itself
15:   upon  $\langle \text{accept}, id, \text{val}, \text{bf} \rangle$  from server  $L$  do  $\triangleright$  every server, including  $L$  itself
16:    if promised_id  $\leq id$  then
17:     promised_id  $\leftarrow id$ ; accepted_id  $\leftarrow id$ 
18:     accepted_value  $\leftarrow \text{val}$ ; BF  $\leftarrow \text{bf}$ 
19:     send  $\langle \text{accepted}, id \rangle$  to  $L$ 
20:    else
21:     ignore  $\triangleright$  if  $s = L$ , the leader abandons the proposal
22:    end if
23:   upon  $\langle \text{accepted}, id \rangle$  with pending  $\neq \perp$  and  $id = \text{pending.id}$  do
24:    P2accepts  $\leftarrow \text{P2accepts} \cup \{\text{sender}\}$ 
25:    if  $|\text{P2accepts}| = F+1$  then
26:     committed_value  $\leftarrow \text{pending}.n_{\text{val}}$ 
27:     reply pending. $n_{\text{val}}$  to pending.client; pending  $\leftarrow \perp$ 
28:    end if
29:   upon a read request, while state = (leader, accept) and committed_value  $\neq \perp$  do  $\triangleright$  requires a leader lease (Section III)
30:    reply committed_value
```

that promised in Phase 1 (each having set promised_id = (round, 0)). Once a majority of acceptances (including the leader’s own) has been recorded, the leader records n_{val} as committed_value, returns n_{val} to the client, and clears pending (lines 25–27).

Reads: A read command is served locally by the leader from its committed_value, without contacting the followers (lines 29–30). For such a local read to return the latest

committed state, the server serving it must still be the current leader. We assume a leader *lease*: the oracle elects no higher-round leader before the lease expires. Lease-based local reads are a standard technique in leader-based systems, and are discussed for Raft [3]. Because the same assumption applies to Raft and Bloom Paxos alike, Table I lists zero-cost, zero-RTT reads for both.

B. Correctness Properties

We state the protocol’s two guarantees here and prove them in Appendix A. Below, a proposal is *committed* once a majority of servers have accepted it; an *ideal* Bloom filter has no false positives and no false negatives; the *command sequence* of a proposal is the sequence of client commands the protocol executed to produce its state; and $\text{apply}(\sigma, s_0)$ denotes applying the commands of σ , in order, to the initial state s_0 .

Theorem (At-Most-Once Execution). *In every execution of Bloom Paxos with an ideal Bloom filter, there is a single sequence σ_{exec} of client-submitted commands such that:*

- 1) (Agreement) *the command sequence of every committed proposal is a prefix of σ_{exec} , and every committed state equals $\text{apply}(\sigma, s_0)$ for such a prefix σ ;*
- 2) (At-most-once) *each opid occurs at most once in σ_{exec} ;*
- 3) (Real-time order) *if operation o_1 receives its response before operation o_2 is invoked, then o_1 ’s command precedes o_2 ’s command in σ_{exec} .*

An operation spans its *invocation*, when its client first sends $\langle C, \text{opid} \rangle$, and its *response*, when that client receives a reply for *opid*; retries lie between the two and do not start a new operation. In (iii), o_1 and o_2 may come from the same or from different clients, and “before” refers to real time; operations whose intervals overlap are unconstrained.

Theorem (Liveness). *In an execution of Bloom Paxos with $N = 2F + 1$ server nodes, at most F of which may crash, and reliable channels, suppose the oracle eventually produces a round r satisfying: (1) the designated leader L for round r does not fail; (2) at least F other nodes do not fail (so L plus these nodes form a majority); and (3) while L processes the operations of interest, the oracle emits no leader message with round $r' > r$. Then every client operation submitted to L during round r eventually receives a response.*

C. Performance and Cost Analysis

Our goal is to compare Bloom Paxos against log-replicating state-machine protocols in terms of data transfer cost and operation latency, and to identify the regimes in which one outperforms the other. We take Raft as the representative baseline: Multi-Paxos [1] and Sequence Paxos [15] share the same first-order data-transfer cost, since they too replicate commands, incurring cost proportional to the command size per replica, and must transfer missing log entries when the quorum changes. We use Raft concretely because, unlike Multi-Paxos, it admits no log holes, giving a clean per-command accounting; the comparison then carries over to

these other log-based protocols. Our expressions use a parsimonious model based on the following system parameters:

- c : the size of a single command.
- s : the size of the state machine’s state.
- N : the number of nodes in the system.
- b : the size of the 2-BF (Section II)
- ρ : an upper bound on the number of quorum/leader changes per command; that is, any t successive commands incur at most ρt quorum/leader changes.
- D : the number of log entries a recovering or newly joined node must fetch per quorum change (the catch-up backlog).

Our model shows that Bloom Paxos offers a trade-off between accuracy and data transfer cost. The read and RMW latencies are comparable in both protocols; which approach offers a lower long-term cost depends on the relative sizes of $(N - 1)(1 + \rho D)/2 \cdot c$ (Raft) and $(N - 1)(1 + \rho)/2 \cdot (s + b)$ (Bloom Paxos). Higher accuracy requires a larger Bloom filter, and therefore more data transfer. Because Bloom Paxos transmits state rather than commands, if the state grows as new commands are applied, the long-term cost of Bloom Paxos can eventually exceed that of log-transmitting protocols such as Raft. We validate this first-order model experimentally in Section IV-C, where our prototype exhibits a 43% data-transfer reduction consistent with these expressions.

D. Applicability

Bloom Paxos transfers only the current state and a constant-size Bloom filter, never the command history; a log-based protocol must transfer the history in full. Bloom Paxos is therefore advantageous when the application’s state stays small as commands accumulate, and when reconfigurations are frequent enough that transferring a growing command-log costs more than transferring the state and Bloom filter. Formally, this is the regime where Bloom Paxos’s long-term per-command cost $(1 + \rho)(s + b)$ is smaller than Raft’s $(1 + \rho D) c$ (the shared factor $(N - 1)/2$ cancels); since the backlog D grows over time, the ρD term makes Raft the more expensive protocol as churn and command history increase. In short, favor Bloom Paxos when the state stays small and bounded while a command log would accumulate without bound.

Favorable scenarios. Bloom Paxos is well-suited to applications whose state machine has a *small, fixed-size state*, so that transferring the state and Bloom filter on reconfiguration is cheaper than re-shipping an ever-growing log. Examples include:

- *Bounded read-modify-write registers* such as distributed counters, gauges, and configuration flags, where the state is a single value or a small record that stays fixed in size no matter how many operations accumulate.
- *Coordination metadata* such as leader leases, lock ownership, or small membership tables that are updated frequently but remain compact.
- *Systems with frequent quorum or leader changes* (high ρ), where Raft’s need to transfer a full log to new nodes is particularly costly (the ρD term in Table I), while Bloom Paxos only transfers the current state and Bloom filter.

	Raft	Bloom Paxos
Cost of a RMW command	$c \cdot (N - 1)/2$	$(s + b) \cdot (N - 1)/2$
Cost of a read command	0	0
Long-term overall cost per command inclusive of quorum/leader changes	$(N - 1)(1 + \rho D)/2 \cdot c$	$(N - 1)(1 + \rho)/2 \cdot (s + b)$
Read latency (RTT)	0	0
RMW latency (RTT)	1	1

TABLE I

FIRST-ORDER MODEL OF DATA-TRANSFER COST AND LATENCY FOR BLOOM PAXOS VS. RAFT. THROUGHOUT, $cost$ DENOTES THE VOLUME OF DATA TRANSFERRED (E.G., BYTES PER COMMAND), NOT COMPUTATION OR STORAGE. LATENCIES ARE EXPRESSED AS RTTs BETWEEN THE LEADER AND THE FOLLOWERS. WE ASSUME CLIENT-LEADER LATENCY IS 0.

Unfavorable scenarios. Bloom Paxos is less suitable when the state grows unboundedly with the number of commands (e.g., a key-value store accumulating entries): every accept message then carries a payload of size $s + b$ that grows with the history, whereas a log-based protocol transmits a fixed-size command per operation. It is also a poor fit for applications that require a command log for purposes beyond consensus, such as auditing, debugging, event replay, or external change-data capture, since Bloom Paxos deliberately eliminates explicit logs.

IV. EVALUATION

We evaluate Bloom Paxos using a combination of prototype-based and numerical experiments. We implemented a prototype distributed system in C++ based on Bloom Paxos, using gRPC for inter-node communication. We evaluate its performance by comparing it with a publicly available implementation of Raft [28].

A. Bloom Filter False Positive Rate

To measure the false positive rate (FPR) of our 2-BF, we insert 10 million unique elements one at a time. Before each insertion, we query the 2-BF for the element; if the query reports membership, we treat the element as a “duplicate” and do not insert it. The empirical FPR is the fraction of elements flagged as duplicates. Figure 2 compares the empirically observed FPR with the theoretical estimate. The observed error remains consistently below the theoretical bound, suggesting that practical implementations of Bloom Paxos can offer predictable FPR guarantees.

B. A Counter Implementation

We evaluate the accuracy of a distributed counter implemented using Bloom Paxos. The goal is to quantify the loss of increments caused by false positives in the Bloom-filter-based duplicate-detection mechanism. We execute 10 million increment operations and record the final counter value returned by the system. Configured with a target false-positive probability $p_{2bf} = 10^{-8}$, Bloom Paxos introduces **no counter error** across all runs. We also repeated the experiment with multiple Bloom-filter reset sizes; in every case, the final counter value matched the expected 10 million. For practical and reasonably small false-positive targets, Bloom Paxos thus preserves application-level semantics without introducing counting error.

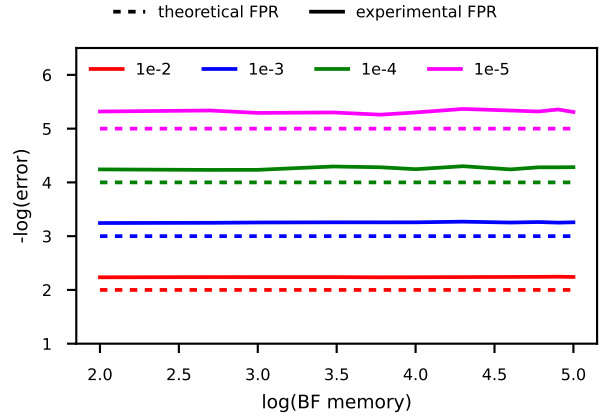


Fig. 2. Observed FPR of the 2-BF. Here n is the number of insertions allowed between successive resets of the 2-BF.

C. Comparison With Raft

Finally, we use our prototype to compare long-term cost and operation latency against Raft. To highlight the savings in data transfer, we replay the same workload under both Raft and Bloom Paxos. We simulate a user application whose requests arrive as a Poisson process with a mean rate of 60 req/min, each request targeting a 20 KB object. Each experiment runs for 15 minutes on an emulated geo-distributed system across 5 locations (Tokyo, Frankfurt, Sydney, London, and São Paulo) with quorums of size 3. To emulate the geo-distributed setting, we inject inter-node delays drawn from inter-site latencies reported for these locations on Google Cloud Platform [29]. We trigger a quorum change once every 5 minutes. The 2-BF is 2.693 KB in size.

Under Raft, each reconfiguration (quorum change) forces the leader to transfer to the new node all data it lacks. This step is critical: the leader cannot proceed with new client requests until the data is acknowledged by a majority of nodes. These transfers appear as the red-colored segments in Figure 3.

Under Bloom Paxos, adding a new node requires no additional data transfer from the leader: the leader simply sends the current state and the Bloom filter. Overall, Bloom Paxos reduces data-transfer cost by 43% in this setting (Figure 3).

Request latency under Bloom Paxos closely tracks that of Raft—i.e., the first-order model in Table I is a good comparative approximation. Moreover, under Raft, the leader is

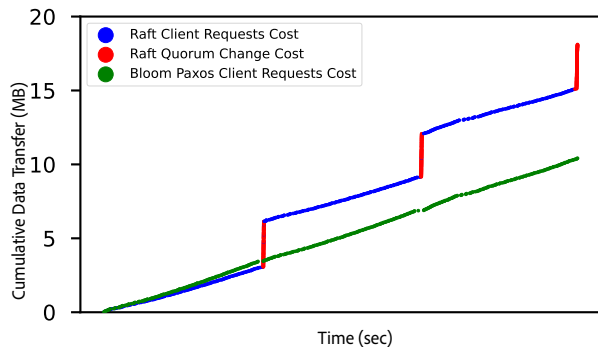


Fig. 3. Data-transfer cost comparison between Bloom Paxos and Raft for the experiment in Section IV-C.

busy transferring data to newly added nodes during a quorum change, delaying the processing of incoming client requests.

V. CONCLUSION

We presented Bloom Paxos, a consensus algorithm that reduces data transfer costs in geo-distributed settings by replacing log replication with direct state replication. The key challenge in this approach is preventing duplicate command execution, which we address with a Bloom filter. The result is a cost-accuracy trade-off governed by the Bloom filter's false-positive rate, which we analyzed formally and evaluated empirically. We proved that, under an ideal Bloom filter, Bloom Paxos executes every client command at most once (Appendix A). Our prototype experiments show that it reduces data transfer costs by 43% over Raft in a geo-distributed deployment with periodic quorum changes, while maintaining comparable latency. As discussed in Section III, Bloom Paxos is best suited to applications whose state is small and stays fixed in size no matter how many commands accumulate, such as distributed counters, configuration registers, and co-ordination metadata. Future work includes hybrid designs that combine state-based replication for the common case while also generating command logs needed for purposes beyond consensus.

REFERENCES

- [1] L. Lamport, "Paxos made simple," *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001), pp. 51–58, 2001.
- [2] —, "The part-time parliament," *ACM Trans. Comput. Syst.*, vol. 16, no. 2, p. 133–169, may 1998. [Online]. Available: <https://doi.org/10.1145/279227.279229>
- [3] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *2014 USENIX annual technical conference (USENIX ATC 14)*, 2014, pp. 305–319.
- [4] I. Moraru, D. G. Andersen, and M. Kaminsky, "There is more consensus in egalitarian parliaments," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, 2013, pp. 358–372.
- [5] Y. Mao, F. P. Junqueira, and K. Marzullo, "Mencius: building efficient replicated state machines for wans," in *OSDI*, vol. 8, 2008, pp. 369–384.
- [6] F. P. Junqueira, B. C. Reed, and M. Serafini, "Zab: High-performance broadcast for primary-backup systems," in *2011 IEEE/IFIP 41st International Conference on Dependable Systems & Networks (DSN)*. IEEE, 2011, pp. 245–256.
- [7] H. Zare, V. R. Cadambe, B. Urgaonkar, N. Alfares, P. Soni, C. Sharma, and A. A. Merchant, "Legostore: a linearizable geo-distributed store combining replication and erasure coding," *Proc. VLDB Endow.*, vol. 15, no. 10, p. 2201–2215, jun 2022. [Online]. Available: <https://doi.org/10.14778/3547305.3547323>
- [8] A. Ailijiang, A. Charapko, M. Demirbas, and T. Kosar, "Wpaxos: Wide area network flexible consensus," 2019.
- [9] Y. L. Chen, S. Mu, J. Li, C. Huang, J. Li, A. Ogus, and D. Phillips, "Giza: Erasure coding objects across global data centers," in *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, 2017, pp. 539–551.
- [10] M. Uluyol, A. Huang, A. Goel, M. Chowdhury, and H. V. Madhyastha, "Near-optimal latency versus cost tradeoffs in geo-distributed storage," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020, pp. 157–180.
- [11] S. Mu, K. Chen, Y. Wu, and W. Zheng, "When paxos meets erasure code: Reduce network and storage cost in state machine replication," in *Proceedings of the 23rd international symposium on High-performance parallel and distributed computing*, 2014, pp. 61–72.
- [12] P. Bailis, S. Venkataraman, M. J. Franklin, J. M. Hellerstein, and I. Stoica, "Probabilistically bounded staleness for practical partial quorums," *arXiv preprint arXiv:1204.6082*, 2012.
- [13] D. Malkhi, M. Reiter, and R. Wright, "Probabilistic quorum systems," in *Proceedings of the sixteenth annual ACM symposium on Principles of distributed computing*, 1997, pp. 267–273.
- [14] F. B. Schneider, "Implementing fault-tolerant services using the state machine approach: A tutorial," *ACM Computing Surveys (CSUR)*, vol. 22, no. 4, pp. 299–319, 1990.
- [15] S. Haridi, L. Kroll, and P. Carbone, "Lecture notes on leader-based sequence paxos – an understandable sequence consensus algorithm," 2020.
- [16] D. Rystsov, "Caspaxos: Replicated state machines without logs," 2018.
- [17] J. Skrzypczak, F. Schintke, and T. Schütt, "Rmwpaxos: Fault-tolerant in-place consensus sequences," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 10, pp. 2392–2405, 2020.
- [18] M. Burrows, "The chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th symposium on Operating systems design and implementation*, 2006, pp. 335–350.
- [19] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 1–22, 2013.
- [20] J. Baker, C. Bond, J. C. Corbett, J. Furman, A. Khorlin, J. Larson, J.-M. Leon, Y. Li, A. Lloyd, and V. Yushprakh, "Megastore: Providing scalable, highly available storage for interactive services," in *CIDR*, vol. 11, 2011, pp. 223–234.
- [21] T. D. Chandra, R. Griesemer, and J. Redstone, "Paxos made live: an engineering perspective," in *Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing*, 2007, pp. 398–407.
- [22] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [23] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: a scalable wide-area web cache sharing protocol," *IEEE/ACM transactions on networking*, vol. 8, no. 3, pp. 281–293, 2000.
- [24] A. Broder and M. Mitzenmacher, "Network applications of bloom filters: A survey," *Internet mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [25] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than bloom," in *Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies*, 2014, pp. 75–88.
- [26] P. S. Almeida, C. Baquero, N. Preguiça, and D. Hutchison, "Scalable bloom filters," *Information Processing Letters*, vol. 101, no. 6, pp. 255–261, 2007.
- [27] R. Subramanyam, I. Gupta, L. M. Leslie, and W. Wang, "Idempotent distributed counters using a forgetful bloom filter," in *2015 International Conference on Cloud and Autonomic Computing*. IEEE, 2015, pp. 113–124.
- [28] R. Siemens, "nidus," <https://github.com/rsiemens/nidus.git>, 2021, commit e668310.
- [29] Google Cloud, "Median latency between zone pairs," <https://console.cloud.google.com/net-intelligence/performance/global-dashboard>, 2024, accessed Mar. 2024.

APPENDIX A

CORRECTNESS PROOFS

This appendix proves the two guarantees stated formally in Section III-B: under an ideal Bloom filter, Bloom Paxos applies every client command at most once, in a single global order consistent with real time (Theorem A.19), and it is live under standard assumptions (Theorem A.20). The intermediate definitions and lemmas below build up to these theorems, which are restated at the point of proof. Throughout, line references are to Algorithms 1 and 2.

A. System Model

We consider $N = 2F + 1$ server nodes communicating over asynchronous, reliable channels: messages between non-faulty servers are neither lost, duplicated, nor corrupted, though they may be arbitrarily delayed. Up to F servers may fail by crashing. A *majority* (or *quorum*) is any set of at least $F + 1$ servers. Throughout this appendix, *node* always means a server node; clients are referred to explicitly. A server's messages to itself are delivered locally and processed before any response to them can arrive; in particular, a leader processes its own *prepare* and *accept* messages before collecting acknowledgments, so its own acknowledgment is always among those it counts. The leader-election oracle is assumed to satisfy the following, as stated in Section III.

Assumption A.1 (Leader-Election Oracle). *The oracle emits messages of the form $\langle \text{Leader}, L, r \rangle$, where L is a server and r a round number, such that:*

- 1) (Uniqueness) *any two oracle messages $\langle \text{Leader}, L, r \rangle$ and $\langle \text{Leader}, L', r \rangle$ carrying the same round r satisfy $L = L'$; that is, at most one server is designated leader in any round;*
- 2) (Monotonicity) *the round numbers of successive oracle messages are strictly increasing.*

Client model. An arbitrary number of client processes interact with the servers. Recall from Section III that a client generates a globally unique operation identifier *opid* for each new command C , and sends $\langle C, \text{opid} \rangle$ to the server it believes to be the leader. Every retry of the same command carries the *same opid*, and distinct commands, whether from the same or different clients, carry distinct *opids*. Clients may fail by crashing; a crashed client's command may go unanswered, but client failures do not affect the safety guarantees below. Client-server channels satisfy the same reliability assumption as server-server channels.

Every node, including a leader, applies the same two acceptor rules: it *promises* a prepare only if the prepare message's ID is strictly greater than its `promised_id` (Alg. 1, line 24), and it *accepts* a proposal only if the accept message's ID is greater than or equal to its `promised_id` (Alg. 2, line 16). A leader's self-promise and self-acceptance pass through these same checks, so every invariant below holds uniformly at every node.

B. Definitions

Definition A.2 (ID and Ordering). *An ID is a pair (r, k) of a round number $r \geq 1$ and a command index $k \geq 0$, ordered lexicographically:*

$$(r_1, k_1) > (r_2, k_2) \iff r_1 > r_2, \text{ or } (r_1 = r_2 \text{ and } k_1 > k_2).$$

Prepare messages carry IDs of the form $(r, 0)$; accept messages carry IDs with $k \geq 1$. Recall from the protocol (Section III; Alg. 1, line 4) that `promised_id` and `accepted_id` are initialized to the distinguished value $\perp$, with $\perp < \text{every ID}$.

Definition A.3 (Proposal). *A proposal is a tuple $P = (id, val, bf)$ broadcast by a leader in an *accept* message (Alg. 2, line 14), where *id* is an ID with index $k \geq 1$, *val* is a state, and *bf* is a Bloom filter. We write $id(P)$, $val(P)$, $bf(P)$ for the components.*

A proposal is identified by its ID: since an ID contains the round number, proposals of different rounds are distinct even if they carry identical values and Bloom filters. Moreover, distinct proposals always carry distinct IDs (see Lemma A.10). Consequently, the lexicographic order on IDs totally orders the proposals of an execution.

Definition A.4 (Committed Proposal). *A proposal P is committed if there exists a majority S of nodes such that every node $s \in S$, at some point in the execution, accepts P , i.e., sets `accepted_value` = $val(P)$ and `accepted_id` = $id(P)$ in response to receiving P (Alg. 2, lines 16–19).*

We remark that, in the above definition, the nodes of S need not accept P at the same time, and each may subsequently overwrite these variables upon accepts with higher IDs.

Definition A.5 (Ideal Bloom Filter). *An ideal Bloom filter answers every membership query exactly: it returns `true` on *opid* if and only if *opid* was previously inserted. It produces no false positives and no false negatives.*

Definition A.6 (Command Sequence). *Assume an ideal Bloom filter. For each proposal P at ID (r, k) we define its command sequence $\sigma(P)$ by induction on the ID ordering, via a predecessor sequence σ_0 and predecessor Bloom filter bf_0 :*

- *If $k \geq 2$: let P_{prev} be the proposal at $(r, k-1)$, issued earlier by the same leader. Set $\sigma_0 = \sigma(P_{prev})$ and $bf_0 = bf(P_{prev})$.*
- *If $k = 1$ and the highest `accepted_id` reported in the leader's Phase 1 acks (Alg. 1, lines 30–33) is $id^* \neq \perp$: let P^* be the unique proposal at id^* . Set $\sigma_0 = \sigma(P^*)$ and $bf_0 = bf(P^*)$. (A node updates `accepted_value`, `accepted_id`, and *BF* together, so the ack reporting id^* carries exactly $val(P^*)$ and $bf(P^*)$, and these are the values the leader adopts at Alg. 1, line 35.)*
- *If $k = 1$ and all Phase 1 acks reported $\perp$: set $\sigma_0 = \langle \rangle$ and $bf_0 = \emptyset$ (empty filter).*

*Let C with identifier *opid* be the command the leader processes for this proposal. The leader queries its Bloom filter*

for *opid* (Alg. 2, line 7), which selects between the two cases below. Then

$$\sigma(P) = \begin{cases} \sigma_0 \cdot \langle C \rangle & \text{if } \text{opid} \notin \text{bf}_0 \text{ (Alg. 2, line 8)} \\ \sigma_0 & \text{if } \text{opid} \in \text{bf}_0 \text{ (Alg. 2, line 10)} \end{cases}$$

Here $\text{apply}(\sigma, s_0)$ denotes the sequential application of the commands of σ to the initial state s_0 . Note that this definition introduces only $\sigma(P)$: the value $\text{val}(P)$ and the Bloom filter $\text{bf}(P)$ are components of the proposal itself (Definition A.3), fixed by the execution; how they relate to $\sigma(P)$ is a statement to be proved (Lemma A.9).

Definition A.6 is recursive, so it defines $\sigma(P)$ only once we know that the proposals of an execution can be enumerated in an order along which the recursion may proceed. This is what the next two lemmas establish: proposals are totally ordered by their IDs, and each proposal's defining case refers only to a strictly earlier proposal in that order.

Lemma A.7 (Total Order of Proposals). *The proposals issued in an execution are totally ordered by their IDs, and this order is a well-order: every non-empty set of proposals has a least element.*

Proof. By Lemma A.10 distinct proposals carry distinct IDs, so ordering proposals by ID is well-defined and antisymmetric. IDs are pairs (r, k) of integers with $r \geq 1$ and $k \geq 0$ compared lexicographically (Definition A.2); the lexicographic product of two well-ordered sets of non-negative integers is itself a total well-order. $\square$

Lemma A.8 (Well-Definedness of the Command Sequence). *Definition A.6 assigns a unique command sequence $\sigma(P)$ to every proposal P .*

Proof. By Lemma A.7 we may define σ on the proposals of an execution sequentially, in increasing order of ID, provided that each proposal's defining case is unambiguous and refers only to a proposal of strictly smaller ID. We verify both.

Unambiguous. Let P have ID (r, k) ; by Definition A.3, $k \geq 1$. Either $k \geq 2$, or $k = 1$ and the highest accepted_id in the leader's Phase-1 acks is some $\text{id}^* \neq \perp$, or $k = 1$ and all acks reported $\perp$: exactly one case applies. In the first two cases the case refers to the proposal at $(r, k-1)$ and at id^* respectively, each unique by Lemma A.10; each also exists, since the leader of round r increments cmd_index by one per proposal (Alg. 2, line 12), and since a node reports $\text{accepted_id} = \text{id}^* \neq \perp$ only after accepting a proposal at id^* .

Strictly earlier. For $k \geq 2$, $(r, k-1) < (r, k)$. For $k = 1$, $\text{id}^* < (r, 0) < (r, 1)$ by Lemma A.13. The third case is a base case and refers to no proposal.

Each $\sigma(P)$ is therefore determined by the values of σ already assigned to proposals of strictly smaller ID, so proceeding along the well-order of Lemma A.7 assigns each proposal exactly one command sequence. (Lemmas A.10 and A.13 are proved from the oracle assumption and the acceptor rules alone and never mention σ , so this argument is not circular.) $\square$

Lemma A.9 (Proposal Contents). *Let P be a proposal at ID (r, k) with predecessor data σ_0, bf_0 as in Definition A.6, and let *opid* be the identifier of the command the leader processes for P . Then*

$$\text{bf}(P) = \begin{cases} \text{bf}_0 \cup \{\text{opid}\} & \text{if } \text{opid} \notin \text{bf}_0 \\ \text{bf}_0 & \text{if } \text{opid} \in \text{bf}_0 \end{cases}$$

and $\text{val}(P) = \text{apply}(\sigma(P), s_0)$. Moreover, at the moment the leader forms P its local BF equals bf_0 and its local accepted_value equals $\text{apply}(\sigma_0, s_0)$.

Proof. Throughout, we use that the leader accepts no proposal other than its own between the step that fixes its predecessor—its acceptance of the proposal at $(r, k-1)$ if $k \geq 2$, its Phase-1 adoption if $k = 1$ —and the step that forms P . Indeed, both of those steps leave $\text{promised_id} = (r, \text{cmd_index})$, whereas accepting any other proposal would raise promised_id strictly above that value (Lemma A.12), and the guard of the proposing rule (Alg. 2, line 5) then prevents P from being issued at all.

The leader forms P from its local accepted_value and BF: it sets $\text{BF}_{\text{new}} = \text{BF} \cup \{\text{opid}\}$ and $n_{\text{val}} = \text{apply}(C, \text{accepted_value})$ if $\text{opid} \notin \text{BF}$ (Alg. 2, line 8), and $\text{BF}_{\text{new}} = \text{BF}$, $n_{\text{val}} = \text{accepted_value}$ otherwise (Alg. 2, line 10); it then broadcasts $\langle \text{accept}, \text{id}, n_{\text{val}}, \text{BF}_{\text{new}} \rangle$, so $\text{val}(P) = n_{\text{val}}$ and $\text{bf}(P) = \text{BF}_{\text{new}}$ (Alg. 2, line 14). It therefore suffices to show that at that moment $\text{BF} = \text{bf}_0$ and $\text{accepted_value} = \text{apply}(\sigma_0, s_0)$; the displayed equations then follow by the case split of Definition A.6, and the claim about $\text{val}(P)$ follows by induction along the order of Lemma A.7, since $\text{apply}(\sigma_0 \cdot \langle C \rangle, s_0) = \text{apply}(C, \text{apply}(\sigma_0, s_0))$.

Case $k \geq 2$. The leader broadcast P_{prev} at $(r, k-1)$ to all servers including itself and, by local delivery, processed its own copy before any response; passing the acceptance check it set $\text{accepted_value} = \text{val}(P_{\text{prev}})$ and $\text{BF} = \text{bf}(P_{\text{prev}}) = \text{bf}_0$ (Alg. 2, lines 16–19). By serialization the leader dequeues the next command only after P_{prev} has been answered (Alg. 2, line 5), and by the observation above it accepted no other proposal in the interim, so these values are unchanged when P is formed.

Case $k = 1$, adoption. On completing Phase 1 the leader set $\text{accepted_value} = \text{val}^*$ and $\text{BF} = \text{bf}^*$ from the ack reporting id^* (Alg. 1, line 35). A node updates accepted_value , accepted_id and BF in a single step (Alg. 2, line 17), so that ack carries exactly $\text{val}(P^*)$ and $\text{bf}(P^*) = \text{bf}_0$.

Case $k = 1$, all acks $\perp$. The leader's own ack is among those collected (local delivery), so its $\text{accepted_id} = \perp$; having accepted no proposal, it still holds the initial values $\text{accepted_value} = s_0$ and $\text{BF} = \emptyset = \text{bf}_0$ (Alg. 1, lines 4–6), matching $\sigma_0 = \langle \rangle$. $\square$

Notation. We write $\sigma_1 \preceq \sigma_2$ when σ_1 is a *prefix* of σ_2 , i.e., $\sigma_2 = \sigma_1 \cdot \sigma'$ for some (possibly empty) sequence σ' . For $\sigma = \langle C_1, \dots, C_m \rangle$ we write $\text{opids}(\sigma) = \{\text{opid}(C_1), \dots, \text{opid}(C_m)\}$.

C. Basic Invariants

Lemma A.10 (ID Uniqueness). *For every ID, at most one proposal is issued with that ID.*

Proof. By Assumption A.1(1), each round r has at most one leader. Within round r the leader increments `cmd_index` before stamping each proposal (Alg. 2, line 12), so its proposals carry the distinct indices $1, 2, \dots$ $\square$

Lemma A.11 (Ordering Invariant). *At every node, $\text{accepted_id} \leq \text{promised_id}$ at all times.*

Proof. Initially the invariant holds: every node starts with $\text{promised_id} = \text{accepted_id} = \perp$ (Alg. 1, line 4).

Suppose, for contradiction, that the invariant fails somewhere in the execution, and let β be the *first* point at which it fails, say at node s . Since β is the first violation, the invariant held at *every* node at *every* point strictly before β . The step taken at β must therefore assign to promised_id or accepted_id at s , and exactly three steps of the protocol do so. Write p and a for the values of promised_id and accepted_id at s immediately before β ; by the choice of β , $a \leq p$.

Case 1: a promise (Alg. 1, line 25). This step executes only if $p < id$ (Alg. 1, line 24) and sets $\text{promised_id} \leftarrow id$, leaving accepted_id unchanged. Hence afterwards $\text{accepted_id} = a \leq p < id = \text{promised_id}$.

Case 2: an acceptance (Alg. 2, line 17). This step sets both variables to the same value id , so afterwards $\text{accepted_id} = \text{promised_id}$.

Case 3: Phase-1 adoption (Alg. 1, line 35). Here s is the leader of some round r ; the step sets $\text{accepted_id} \leftarrow \text{aid}^*$ and leaves $\text{promised_id} = p$ unchanged, where aid^* is the largest accepted_id among the $F+1$ acks collected for the prepare $(r, 0)$ (Alg. 1, lines 30–33). We bound both sides.

First, $\text{aid}^* < (r, 0)$. The ack reporting aid^* was sent by some node i that passed the promise check for $(r, 0)$ (Alg. 1, line 24), so at that earlier moment $\text{promised_id}_i < (r, 0)$, and aid^* is the value of accepted_id_i at that same moment (Alg. 1, line 26). That moment precedes β , so the invariant held at i then: $\text{aid}^* = \text{accepted_id}_i \leq \text{promised_id}_i < (r, 0)$.

Second, $p \geq (r, 0)$. Node s sent the prepare $(r, 0)$ to itself (Alg. 1, line 19) and, by the local-delivery assumption, processed it before collecting any ack. After processing it, $\text{promised_id}_s \geq (r, 0)$: either s promised, setting it to exactly $(r, 0)$ (Alg. 1, line 25), or s declined, which by the check at Alg. 1, line 24 requires $\text{promised_id}_s \geq (r, 0)$ already. Moreover promised_id never decreases, since Cases 1 and 2 are the only steps that modify it and both leave it no smaller. Hence $p \geq (r, 0)$ at β .

Combining, $\text{accepted_id} = \text{aid}^* < (r, 0) \leq p = \text{promised_id}$ after the step.

In all three cases the invariant holds immediately after β , contradicting the assumption that it fails at β . Hence the invariant holds at every node at all times. (A leader's self-

promise and self-acceptance pass through the same checks, so no separate case is needed.) $\square$

Lemma A.12 (Monotonicity). *At every node, promised_id and accepted_id are monotonically non-decreasing over time.*

Proof. The claim holds initially, when both variables equal $\perp$ at every node (Alg. 1, line 4).

Suppose it fails somewhere in the execution, and let β be the *first* point at which it fails, at node s ; thus both variables were non-decreasing at every node up to β , and the step taken at β lowers promised_id or accepted_id at s . Only three steps assign these variables. Write p and a for their values immediately before β .

Case 1: a promise (Alg. 1, line 25). The step runs only if $p < id$ (Alg. 1, line 24) and sets $\text{promised_id} \leftarrow id > p$, leaving accepted_id untouched. Neither variable decreases.

Case 2: an acceptance (Alg. 2, line 17). The step runs only if $p \leq id$ (Alg. 2, line 16) and sets both variables to id . Then $\text{promised_id} = id \geq p$, and $\text{accepted_id} = id \geq p \geq a$, the last inequality by Lemma A.11. Neither variable decreases.

Case 3: Phase-1 adoption (Alg. 1, line 35). The step sets $\text{accepted_id} \leftarrow \text{aid}^*$ and leaves promised_id unchanged, so it suffices to show $\text{aid}^* \geq a$. Let r be s 's current round. The handler runs only while $\text{promised_id}_s = (r, 0)$ (Alg. 1, line 30), so $p = (r, 0)$. By local delivery, s answered its own prepare before collecting any ack, at which time it set $\text{promised_id}_s = (r, 0)$ and placed its own accepted_id into `Packs`. Between that moment and β , node s performed no acceptance: an acceptance sets promised_id to the accepted ID, which has command index at least 1 (Definition A.3) and hence exceeds $(r, 0)$, contradicting $p = (r, 0)$ together with the non-decrease of promised_id established in Cases 1 and 2. Nor did s perform another adoption for round r , since the handler adopts only on the single step at which $|\text{Packs}|$ reaches $F+1$ (Alg. 1, line 32). Hence accepted_id_s is unchanged since that ack, so the entry s contributed to `Packs` carries exactly a . As aid^* is the largest aid in `Packs` (Alg. 1, line 33), $\text{aid}^* \geq a$.

In every case neither variable decreases at β , contradicting the choice of β . Hence promised_id and accepted_id are non-decreasing at every node at all times. $\square$

Lemma A.13 (Phase 1 Response Bound). *If a node acks a prepare for round r (carrying ID $(r, 0)$), then the accepted_id it reports satisfies $\text{accepted_id} < (r, 0)$. In particular, the reported proposal belongs to a round earlier than r .*

Proof. A node acks only when $\text{promised_id} < (r, 0)$. By Lemma A.11, $\text{accepted_id} \leq \text{promised_id} < (r, 0)$. $\square$

Lemma A.14 (Quorum Intersection). *Any two majorities of $N = 2F+1$ nodes share at least one node.*

Proof. Two disjoint majorities would require $\geq 2(F+1) = 2F+2 > N$ nodes. $\square$

Lemma A.15 (BF–Sequence Consistency). *Under an ideal Bloom filter, every proposal P satisfies $bf(P) = \text{opids}(\sigma(P))$.*

Proof. The proof is by induction on $id(P)$ along the well-order of Lemma A.7; the inductive hypothesis (IH) is that $bf(Q) = \text{opids}(\sigma(Q))$ for every proposal Q with $id(Q) < id(P)$.

Fix a proposal P at ID (r, k) , and let C with identifier $opid$ be the command its leader processes. Recall from Definition A.6 that σ_0 and bf_0 denote the predecessor command sequence and Bloom filter of P : the pair from which $\sigma(P)$ is built. Definition A.6 fixes them by three cases, and we first show that

$$bf_0 = \text{opids}(\sigma_0) \quad (1)$$

holds in each of them.

Case (a): $k \geq 2$. Here $\sigma_0 = \sigma(P_{prev})$ and $bf_0 = bf(P_{prev})$, where P_{prev} is the proposal at $(r, k-1)$. Since $(r, k-1) < (r, k)$, the IH applies to P_{prev} and gives (1).

Case (b): $k = 1$ and the highest accepted_id in the leader's Phase-1 acks is $id^ \neq \perp$.* Here $\sigma_0 = \sigma(P^*)$ and $bf_0 = bf(P^*)$ for the proposal P^* at id^* . Since $id^* < (r, 0) < (r, 1)$ (Lemma A.13), the IH applies to P^* and gives (1).

Case (c): $k = 1$ and every Phase-1 ack reported $\perp$. This is the base case: no ack refers to an earlier proposal, so there is nothing to appeal to, and Definition A.6 sets $\sigma_0 = \langle \rangle$ and $bf_0 = \emptyset$. These choices are the correct ones because $\text{accepted_id} = \perp$ at every acking node means no such node has ever accepted a proposal; in particular the leader's own ack reports $\perp$ (local delivery), so it still holds its initial $\text{accepted_value} = s_0$ and empty BF (Alg. 1, lines 4–6), as stated in Lemma A.9. Since $\text{opids}(\langle \rangle) = \emptyset$, (1) holds.

Now we pass from the predecessor to P itself. When the leader forms P its local BF equals bf_0 (Lemma A.9), and it tests whether $opid \in \text{BF}$ (Alg. 2, line 7). Because the filter is ideal (Definition A.5), this test returns `true` exactly when $opid$ belongs to the set bf_0 ; the branch the algorithm takes therefore coincides with the case split of Definition A.6. Two cases remain.

If $opid \notin bf_0$, the leader appends the command and inserts the identifier: $\sigma(P) = \sigma_0 \cdot \langle C \rangle$ (Definition A.6) and $bf(P) = bf_0 \cup \{opid\}$ (Lemma A.9; Alg. 2, line 8). Hence

$$\text{opids}(\sigma(P)) = \text{opids}(\sigma_0) \cup \{opid\} \stackrel{(1)}{=} bf_0 \cup \{opid\} = bf(P).$$

If $opid \in bf_0$, the leader treats C as a duplicate and changes neither: $\sigma(P) = \sigma_0$ and $bf(P) = bf_0$ (Lemma A.9; Alg. 2, line 10). Hence $\text{opids}(\sigma(P)) = \text{opids}(\sigma_0) = bf_0 = bf(P)$ by (1).

In both cases $bf(P) = \text{opids}(\sigma(P))$, completing the induction. $\square$

Lemma A.16 (No Duplicates Within a Sequence). *For every proposal P , the opids of the commands in $\sigma(P)$ are pairwise distinct.*

Proof. Induction on $id(P)$. The predecessor sequence σ_0 is either empty or the sequence of an earlier proposal, hence duplicate-free by the inductive hypothesis. If P appends its command C , then $opid(C) \notin bf_0$ (Definition A.6), and $bf_0 = \text{opids}(\sigma_0)$ (Lemma A.15); hence $opid(C)$ does not already occur in σ_0 . $\square$

D. Safety

Lemma A.17 (Intra-Round Prefix). *Let P_j and P_k be proposals of the same round r at IDs (r, j) and (r, k) with $j \leq k$. Then $\sigma(P_j) \preceq \sigma(P_k)$.*

Proof. The proposals of round r are issued by its unique leader one at a time, at indices $1, 2, \dots$ by the serialization rule (Alg. 2, line 5), a new command is dequeued only after the previous proposal has been accepted by a majority and answered. By Definition A.6, each $\sigma(P_{i+1})$ equals $\sigma(P_i)$ or $\sigma(P_i)$ extended by one command, so $\sigma(P_i) \preceq \sigma(P_{i+1})$; transitivity gives the claim. $\square$

We now prove the central safety property.

Theorem A.18 (Prefix Invariant). *In every execution of Bloom Paxos with an ideal Bloom filter, if proposal P_v is committed at ID id_v , then every proposal P with $id(P) > id_v$ satisfies $\sigma(P_v) \preceq \sigma(P)$.*

Proof. By strong induction on $id = id(P)$: assume the claim holds for every proposal P' with $id_v < id(P') < id$ (the inductive hypothesis, IH). Write $id = (r, k)$ and $id_v = (r_v, k_v)$.

Case 1: same round ($r = r_v$, so $k > k_v$). Both proposals were issued by the leader of round r ; Lemma A.17 gives $\sigma(P_v) \preceq \sigma(P)$ directly.

Case 2: higher round, first proposal ($r > r_v$, $k = 1$). P is the first proposal of round r , so its leader completed Phase 1 with acks from a majority Q . Since P_v is committed, some majority Q_v accepted it. By Lemma A.14 there is a node $i \in Q \cap Q_v$.

We claim node i accepted P_v before it sent its Phase 1 ack for round r . Suppose instead the ack came first. Sending the ack set $\text{promised_id}_i = (r, 0)$, which never decreases thereafter (Lemma A.12). Since id_v has round $r_v < r$, we have $id_v < (r, 0)$, so the acceptance check would reject P_v 's accept message — contradicting $i \in Q_v$.

Hence i had already set $\text{accepted_id}_i = id_v$ when it acked, and by Lemma A.12 its ack reports $\text{accepted_id} \geq id_v$. The highest reported ID id^* therefore satisfies

$$id_v \leq id^* < (r, 0) < id,$$

using Lemma A.13 for the upper bound. Let P^* be the unique proposal at id^* (Lemma A.10); by Definition A.6, $\sigma_0 = \sigma(P^*)$ and $\sigma(P^*) \preceq \sigma(P)$.

- If $id^* = id_v$: then $P^* = P_v$, so $\sigma(P_v) = \sigma_0 \preceq \sigma(P)$.
- If $id^* > id_v$: the IH applies to P^* , whose ID lies strictly between id_v and id (note the IH does not require P^* to be committed), giving $\sigma(P_v) \preceq \sigma(P^*) \preceq \sigma(P)$.

Case 3: higher round, subsequent proposal ($r > r_v, k \geq 2$). Let P_{prev} be the proposal at $(r, k-1)$. Since $id_v < (r, k-1) < id$, the IH gives $\sigma(P_v) \preceq \sigma(P_{prev})$, and Definition A.6 gives $\sigma(P_{prev}) \preceq \sigma(P)$. $\square$

Theorem A.19 (At-Most-Once Execution). *In every execution of Bloom Paxos with an ideal Bloom filter there is a single, duplicate-free sequence σ_{exec} of client-submitted commands such that:*

- 1) (Agreement) *the command sequence of every committed proposal is a prefix of σ_{exec} , and every committed state equals $\text{apply}(\sigma, s_0)$ for such a prefix σ ;*
- 2) (At-most-once) *each $opid$ occurs at most once in σ_{exec} ;*
- 3) (Real-time order) *if operation o_1 receives its response before operation o_2 is invoked, then o_1 's command precedes o_2 's command in σ_{exec} .*

Here an operation o spans two events: its invocation, the point at which its client first sends $\langle C, opid \rangle$, and its response, the point at which that client receives a reply for $opid$; any retries of o carry the same $opid$ and lie between these two events rather than starting a new operation. In (3), o_1 and o_2 may be issued by the same client or by different clients, and “before” refers to real time: the response event of o_1 occurs at an earlier time than the invocation event of o_2 . Operations whose intervals overlap are unconstrained by (3).

Proof. (1) By Theorem A.18, the command sequences of committed proposals are pairwise ordered by $\preceq$: for any two committed proposals, the one with the smaller ID has the prefix. Let σ_{exec} be the union of this chain (its longest element, or its limit if the execution is infinite). Every committed state equals $\text{apply}(\sigma(P), s_0)$ by Lemma A.9. Every command in every $\sigma(P)$ was dequeued from a leader's `cmd_queue`, which is populated only by client messages (Alg. 2, lines 1–3), so all commands in σ_{exec} were submitted by clients.

(2) Each $\sigma(P)$ is duplicate-free (Lemma A.16), and σ_{exec} is the union of a prefix-chain of such sequences, so it is duplicate-free as well.

(3) Let C_1, C_2 be the commands of o_1, o_2 , and let P_1 be the committed proposal whose commitment triggered o_1 's response. First, $C_1 \in \sigma(P_1)$: if the leader executed C_1 in P_1 this is immediate; if the leader detected a duplicate, then $opid(C_1) \in bf_0 = \text{opids}(\sigma_0)$ (Lemma A.15) and $\sigma_0 \preceq \sigma(P_1)$. Second, $C_2 \notin \sigma(P_1)$: every command in $\sigma(P_1)$ was submitted before P_1 was issued, whereas C_2 was first submitted after o_1 's response, which followed P_1 's issuance. Since $\sigma(P_1) \preceq \sigma_{exec}$ and C_2 appears (if at all) only beyond that prefix, C_1 precedes C_2 in σ_{exec} . $\square$

Theorem A.19 states that the replicated state evolves exactly as if a single correct server executed the clients' commands one at a time, applying each at most once despite retries.

E. Liveness

Theorem A.20 (Liveness). *In an execution of Bloom Paxos with $N = 2F+1$ nodes, at most F crash failures, and reliable*

channels, suppose the oracle eventually produces a round r satisfying:

- 1) *the designated leader L for round r does not fail,*
- 2) *at least F other nodes do not fail (so L plus these nodes form a majority), and*
- 3) *during the interval in which L processes the operations of interest, the oracle does not emit any leader message with round $r' > r$.*

Then every client operation submitted to L during round r eventually receives a response.

Proof. **Phase 1.** L broadcasts a prepare with ID $(r, 0)$. By condition 3 and Assumption A.1(2), every prepare and accept message in the execution so far carries a round smaller than r , so every non-faulty node (including L itself) has `promised_id` $< (r, 0)$ when the prepare arrives; each promises and acks. In particular L promises its own prepare, setting `promised_idL` $= (r, 0)$, and by condition 3 no message of a higher round reaches it during the interval, so `promised_idL` remains $(r, 0)$ and the check of the ack handler (Alg. 1, line 30) passes for every ack. With at least $F+1$ non-faulty nodes, L collects a majority of acks and completes the phase.

Phase 2. For each command, L broadcasts an accept with ID (r, k) . When it arrives at a non-faulty node, that node's `promised_id` is at most (r, k) : by condition 3 it has seen no message of any round above r , and within round r it has at most promised $(r, 0)$ or accepted proposals with indices below k . The node therefore accepts and replies. Counting L 's own acceptance, a majority of acceptances is collected and L responds to the client.

Commands are dequeued and proposed one at a time, each completing as above, so every operation submitted to L eventually receives a response. $\square$